Docker指令大全

一、基本命令

二、镜像管理指令

三、容器管理指令

四、数据管理与卷指令

五、网络配置指令

六、安全与维护指令

七、其他更多指令

一、基本命令

Docker的基本命令是Docker操作的基础，理解这些命令对于任何使用Docker的人来说都至关重要。这些命令涵盖了从容器的生命周期管理到镜像的操作等多方面的功能。在这一部分，我们将以表格的形式详细介绍各个基本命令及其用途。

命令	描述	示例
docker run	创建并启动一个容器	docker run -it ubuntu bash
docker ps	列出当前运行的容器	docker ps
docker ps -a	列出所有容器，包括未运行的	docker ps -a
docker stop	停止一个运行中的容器	docker stop [CONTAINER_ID]
docker start	启动一个已停止的容器	docker start [CONTAINER_ID]
docker restart	重启容器	docker restart [CONTAINER_ID]
docker rm	删除一个或多个容器	docker rm [CONTAINER_ID]
docker pull	从仓库拉取一个镜像	docker pull ubuntu
docker images	列出本地存储的镜像	docker images
docker rmi	删除一个或多个镜像	docker rmi [IMAGE_ID]

docker build	从Dockerfile构建镜像	docker build -t my-image:tag .
docker exec	在运行的容器中执行命令	docker exec -it [CONTAINER_ID] bash
docker logs	获取容器的日志	docker logs [CONTAINER_ID]
docker commit	从修改过的容器创建新的镜像	docker commit [CONTAINER_ID] new-image
docker inspect	获取容器或镜像的详细信息	docker inspect [CONTAINER_ID/IMAGE_ID]
docker network	管理Docker网络	docker network ls
docker volume	管理Docker卷	docker volume ls
docker info	显示Docker系统信息	docker info
docker version	显示Docker版本信息	docker version

二、镜像管理指令

在Docker中，镜像是容器运行的基础。掌握镜像管理命令对于有效使用Docker至关重要。以下表格详细介绍了与Docker镜像管理相关的主要命令，包括镜像的拉取、构建、列表展示、删除等操作。

命令	描述	示例
docker images	列出本地的所有镜像	docker images
docker pull	从镜像仓库拉取指定的镜像	docker pull ubuntu:18.04
docker push	将本地镜像推送到镜像仓库	docker push myrepo/myimage:tag
docker rmi	删除一个或多个本地存储的镜像	docker rmi ubuntu
docker build	使用Dockerfile构建新的镜像	docker build -t myimage .
docker history	显示镜像的历史信息	docker history myimage

docker inspect	显示镜像的详细信息	docker inspect ubuntu
docker tag	为镜像添加一个新的标签	docker tag ubuntu:18.04 myubuntu:latest
docker save	将镜像保存为tar归档文件	docker save myimage > myimage.tar
docker load	从tar归档文件加载镜像	docker load < myimage.tar
docker import	从归档文件创建镜像	docker import mycontainer.tar myimage
docker export	将容器快照导出为归档文件	docker export mycontainer > mycontainer.tar
docker create	创建一个新容器但不启动它	docker create ubuntu:18.04
docker commit	从容器创建镜像	docker commit mycontainer mynewimage
docker diff	显示容器文件系统的更改	docker diff mycontainer
docker search	在Docker Hub搜索镜像	docker search ubuntu

三、容器管理指令

容器是Docker的核心概念之一，有效管理容器对于任何Docker用户都是必备技能。以下是关于Docker容器管理的指令集合。这部分内容以表格形式展现，详细列出了与容器生命周期管理、操作、监控及其他相关功能的主要命令。

命令	描述	示例
docker run	创建一个新容器并运行一个命令	docker run –it ubuntu /bin/bash
docker ps	列出当前运行中的容器	docker ps
docker ps –a	列出所有容器，包括未运行的	docker ps –a
docker stop	停止一个或多个运行中的容器	docker stop [CONTAINER_ID]

docker start	启动一个或多个已停止的容器	docker start [CONTAINER_ID]
docker restart	重启一个或多个容器	docker restart [CONTAINER_ID]
docker kill	立即终止容器的运行	docker kill [CONTAINER_ID]
docker rm	删除一个或多个容器	docker rm [CONTAINER_ID]
docker exec	在运行的容器中执行命令	docker exec –it [CONTAINER_ID] /bin/bash
docker attach	连接到正在运行的容器	docker attach [CONTAINER_ID]
docker cp	从容器中复制文件/目录到主机，反之亦然	docker cp [CONTAINER_ID]:/path/to/file /dest
docker logs	获取容器的日志	docker logs [CONTAINER_ID]
docker inspect	获取容器的详细信息	docker inspect [CONTAINER_ID]
docker diff	检查容器中文件的更改	docker diff [CONTAINER_ID]
docker commit	从容器创建新的镜像	docker commit [CONTAINER_ID] new_image_name
docker pause	暂停容器中所有的进程	docker pause [CONTAINER_ID]
docker unpause	恢复容器中所有被暂停的进程	docker unpause [CONTAINER_ID]
docker port	显示容器的端口映射	docker port [CONTAINER_ID]

docker rename	重命名容器	docker rename [CONTAINER_ID] new_name
docker stats	显示一个或多个容器的实时资源使用情况	docker stats [CONTAINER_ID]
docker top	显示容器内运行的进程	docker top [CONTAINER_ID]
docker update	更新一个或多个容器的配置	docker update --cpu-shares 512 [CONTAINER_ID]
docker wait	阻塞直到容器停止，然后打印其退出代码	docker wait [CONTAINER_ID]
docker network connect	将容器连接到网络	docker network connect [NETWORK] [CONTAINER_ID]
docker network disconnect	将容器从网络中断开	docker network disconnect [NETWORK] [CONTAINER_ID]

四、数据管理与卷指令

在Docker中，数据管理和卷的概念对于持久化和数据共享至关重要。理解并掌握这些命令可以帮助用户有效地管理容器中的数据。以下是关于Docker数据管理与卷操作的指令集合，包括创建、管理卷，以及数据备份和迁移等功能。

命令	描述	示例
docker volume create	创建一个新的卷	docker volume create my_volume
docker volume ls	列出所有卷	docker volume ls
docker volume inspect	显示指定卷的详细信息	docker volume inspect my_volume
docker volume rm	删除一个或多个卷	docker volume rm my_volume

docker volume prune	删除所有未使用的卷	docker volume prune
docker cp	从主机复制文件到容器，或者从容器复制到主机	docker cp foo.txt mycontainer:/foo.txt
docker create	创建一个新容器，用于数据卷	docker create -v /dbdata --name dbstore training/postgres
docker run -v	在运行容器时挂载卷	docker run -d -P --name web -v /webapp training/webapp python app.py
docker run --mount	使用更详细的挂载配置运行容器	docker run --mount source=my_volume,target=/data my_image
docker service create	在服务中使用卷	docker service create --replicas=1 --name my_service --mount source=my_volume,target=/data my_image

五、网络配置指令

Docker的网络配置是容器化环境中的关键部分，它允许容器之间以及容器与外部世界的通信。理解并掌握网络配置指令对于确保容器应用的正确运行非常重要。以下是关于Docker网络配置的指令集合，这些指令涵盖了网络的创建、管理和调试等方面。

命令	描述	示例
docker network create	创建一个新的网络	docker network create --driver bridge my_network
docker network ls	列出所有网络	docker network ls
docker network rm	删除一个或多个网络	docker network rm my_network

docker network inspect	显示一个或多个网络的详细信息	docker network inspect my_network
docker network connect	连接一个容器到网络	docker network connect my_network my_container
docker network disconnect	断开容器与网络的连接	docker network disconnect my_network my_container
docker run --network	在特定网络下运行一个新的容器	docker run --network=my_network my_image
docker network prune	删除所有未使用的网络	docker network prune
docker service create	在特定网络下创建一个服务	docker service create --network my_network my_service

六、安全与维护指令

在Docker环境中，确保容器安全和进行有效的维护是非常重要的。了解和使用相关的安全与维护指令可以帮助用户保护其容器环境免受未授权访问和潜在威胁，同时确保系统的稳定运行。以下是Docker安全与维护相关的主要指令集合，包括日志记录、系统状态检查和安全扫描等功能。

命令	描述	示例
docker logs	获取容器的日志	docker logs [CONTAINER_ID]
docker stats	显示一个或多个容器的实时资源使用情况	docker stats
docker system df	显示Docker磁盘使用情况	docker system df
docker system prune	清理未使用的资源	docker system prune
docker system events	实时获取Docker系统的事件	docker system events
docker system info	显示系统范围的信息	docker system info

docker inspect	显示容器或镜像的详细信息	docker inspect [CONTAINER_ID or IMAGE_ID]
docker history	显示镜像的历史信息	docker history [IMAGE_ID]
docker checkpoint create	创建容器的检查点	docker checkpoint create [CONTAINER_ID] checkpoint01
docker checkpoint ls	列出容器的检查点	docker checkpoint ls [CONTAINER_ID]
docker checkpoint rm	删除容器的检查点	docker checkpoint rm [CONTAINER_ID] checkpoint01
docker scan	扫描镜像的安全漏洞	docker scan [IMAGE_ID]
docker trust	管理Docker内容信任	docker trust inspect [IMAGE_NAME]
docker secret create	创建一个新的秘密	docker secret create my_secret data.txt
docker secret ls	列出所有秘密	docker secret ls
docker secret rm	删除一个秘密	docker secret rm my_secret
docker swarm init	初始化一个新的Swarm集群	docker swarm init
docker swarm join	加入一个Swarm集群	docker swarm join [OPTIONS] HOST:PORT
docker swarm leave	离开一个Swarm集群	docker swarm leave

七、其他更多指令

Docker提供了一系列丰富的指令，用于处理各种高级场景和特定需求。了解这些更多的指令可以帮助用户更全面地利用Docker的功能，处理特殊的场景或进行高级配置。以下是一些不那么常用，但在特定情况下非常有用的Docker指令集合。

命令	描述	示例
docker attach	连接到正在运行的容器	docker attach [CONTAINER_ID]
docker commit	从修改过的容器创建新镜像	docker commit [CONTAINER_ID] mynewimage
docker diff	检查容器中文件的变化	docker diff [CONTAINER_ID]
docker events	从Docker守护进程获取实时事件	docker events
docker export	导出容器的文件系统为tar归档	docker export [CONTAINER_ID] > mycontainer.tar
docker import	从tar归档创建镜像	docker import myimage.tar
docker load	从tar归档加载镜像	docker load < myimage.tar
docker pause	暂停容器内所有进程	docker pause [CONTAINER_ID]
docker port	列出容器的端口映射	docker port [CONTAINER_ID]
docker rename	重命名容器	docker rename my_container my_new_container
docker save	保存一个镜像为tar归档	docker save myimage > myimage.tar
docker search	在Docker Hub搜索镜像	docker search ubuntu
docker top	显示容器内的运行进程	docker top [CONTAINER_ID]
docker unpause	恢复容器内所有被暂停的进程	docker unpause [CONTAINER_ID]
docker update	更新一个或多个容器的配置	docker update --cpu-shares 512 [CONTAINER_ID]
docker version	显示Docker版本信息	docker version
docker wait	阻塞直到容器停止，然后打印其退出代码	docker wait [CONTAINER_ID]

docker swarm	管理Swarm集群	docker swarm init
docker service	管理Swarm服务	docker service create [OPTIONS] IMAGE [COMMAND] [ARG...]
docker node	管理Swarm节点	docker node ls
docker stack	管理Docker栈	docker stack deploy -c docker-compose.yml mystack
docker secret	管理Docker秘密	docker secret create my_secret data.txt
docker config	管理Docker配置	docker config create my_config config.txt
docker plugin	管理Docker插件	docker plugin install rexray/ebs
docker trust	管理Docker内容信任	docker trust inspect [NAME]
docker context	管理Docker上下文	docker context create my_context
docker buildx	构建Docker镜像（实验性功能）	docker buildx build --platform linux/amd64,linux/arm64 .

这些指令使用户能够处理更复杂的Docker任务，如Swarm集群管理、内容信任管理等。通过熟练掌握这些指令，用户可以充分利用Docker提供的高级功能和灵活性。

> 文章内容源自TechLeadCloud，仅用于学习交流分享，侵删